

Enterprise SOC

Manual de instalación, operación y seguridad

NOC/SOC de monitoreo de infraestructura Windows: telemetría, backups, alertas y auditoría

Versión del documento: 23/09/2026

Uso interno — contiene procedimientos de despliegue y notas de seguridad

Índice

1. Qué es Enterprise SOC
2. Arquitectura del sistema
3. Instalación del NOC/SOC central (VPS Linux)
4. Instalación del agente en cada servidor cliente
5. Seguridad: qué cubre y qué no
6. Operación diaria
7. Problemas conocidos y cómo se resolvieron
8. Checklist final antes de anunciar la URL
9. Referencia rápida de comandos

1. Qué es Enterprise SOC

Enterprise SOC es un sistema propio de monitoreo tipo NOC/SOC para infraestructura Windows (Windows Server y Windows 10/11). Un componente central (backend + dashboard web) recibe datos de agentes livianos instalados en cada servidor a monitorear, y muestra el estado de todos ellos en tiempo real, con generación automática de alertas.

Componentes

- **Backend** (Node.js + Express + PostgreSQL): recibe telemetría y estado de backup vía API, evalúa umbrales, genera alertas, y las distribuye en vivo por WebSocket.
- **Frontend** (Next.js): dashboard con pestañas General, Monitoreo, Topología, Logs y Admin.
- **Agente** (Python, compilado a .exe): corre en cada servidor Windows, reporta CPU/RAM/disco, errores del Visor de Eventos, y estado de backup (Windows Server Backup / wbadmin / VSS).

Qué NO es

No es un producto comercial con soporte externo, no tiene certificaciones de seguridad (SOC 2, ISO 27001, etc.), y no fue auditado por un tercero independiente. Es una herramienta construida a medida — la sección 5 detalla honestamente qué protecciones tiene y cuáles le faltan.

2. Arquitectura del sistema

Central (un VPS por cliente/organización):

- PostgreSQL (datos: usuarios, servidores, telemetría, backups, alertas, auditoría)
- Backend API + WebSocket (puerto interno 3000, solo accesible vía Nginx)
- Frontend / dashboard (puerto interno 3001, solo accesible vía Nginx)
- Nginx como reverse proxy con TLS (Let's Encrypt / Certbot)

En cada servidor a monitorear:

- Agente Windows (.exe), corre como tarea programada con privilegios SYSTEM
- Se comunica solo de forma saliente (outbound HTTPS) hacia el backend central — no escucha ningún puerto entrante, no requiere abrir el firewall del servidor monitoreado

ATENCIÓN: Este sistema no es multi-tenant: cada organización/cliente necesita su propio VPS y su propio backend. No hay aislamiento de datos entre distintos clientes dentro de un mismo backend.

3. Instalación del NOC/SOC central (VPS Linux)

El stack central corre completo con Docker Compose (PostgreSQL + backend + frontend). El VPS no necesita Node.js instalado — todo se construye dentro de los contenedores. Nginx corre en el host como reverse proxy con TLS.

3.1 Provisionar el VPS (una sola vez)

```
scp deploy/setup-server.sh root@TU_IP:/root/  
ssh root@TU_IP  
sudo bash /root/setup-server.sh
```

Instala Docker, Nginx, Certbot, crea el usuario socapp sin privilegios, y configura el firewall (ufw) para que solo los puertos 22 (SSH) y 80/443 (Nginx) queden expuestos.

3.2 Subir el código

```
su - socapp  
git clone <url-de-tu-repo-privado> enterprise-soc  
cd enterprise-soc
```

El repositorio se desarrolló localmente con git pero sin remoto configurado — hay que subirlo a un repositorio privado (GitHub/GitLab) o transferir la carpeta por rsync/scp.

3.3 Configurar variables de producción

```
cp .env.example .env  
nano .env
```

Como mínimo, completar con valores nuevos de producción (nunca reutilizar los de desarrollo):

```
POSTGRES_USER=nocsoc_prod  
POSTGRES_PASSWORD=<openssl rand -hex 32>  
JWT_SECRET=<openssl rand -hex 32>  
AGENT_ENROLLMENT_SECRET=<openssl rand -hex 32>  
CORS_ORIGIN=https://noc.tudominio.com  
NEXT_PUBLIC_API_URL=https://noc.tudominio.com  
NEXT_PUBLIC_WS_URL=wss://noc.tudominio.com/ws
```

3.4 Build y arranque

```
docker compose build  
docker compose up -d  
docker compose ps # los 3 servicios deberian quedar "Up"
```

Las migraciones de base de datos se aplican solas al arrancar el backend (prisma migrate deploy, se ejecuta en cada arranque del contenedor y es seguro correrlo repetidas veces).

3.5 Crear el primer usuario administrador

```
docker exec enterprise-soc-backend node prisma/seed.js
```

Guardar el email/contraseña que imprime: no se pueden recuperar después, solo se guarda su hash.

3.6 Nginx + certificado TLS

```
sudo cp deploy/nginx.conf /etc/nginx/sites-available/enterprise-soc
sudo nano /etc/nginx/sites-available/enterprise-soc # reemplazar el dominio
sudo ln -s /etc/nginx/sites-available/enterprise-soc /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d noc.tudominio.com
```

A partir de acá, <https://noc.tudominio.com> sirve el dashboard con certificado válido.

4. Instalación del agente en cada servidor cliente

ATENCIÓN: Requiere ejecutarse como Administrador en el servidor Windows. Es necesario para que la revisión de backups (wbadmin / WMI de Windows Server Backup) pueda leer el estado real.

4.1 Compilar el agente (una sola vez, en cualquier Windows con Python)

```
cd agent
pip install -r requirements.txt
pip install pyinstaller
pyinstaller --onefile --name enterprise-soc-agent ^
--hidden-import=win32timezone --hidden-import=win32api agent.py
```

4.2 Publicarlo en el backend para descarga automática

```
scp dist/enterprise-soc-agent.exe socapp@TU_IP:~/enterprise-soc/backend/downloads/
docker cp ~/enterprise-soc/backend/downloads/enterprise-soc-agent.exe \
enterprise-soc-backend:/app/downloads/
```

4.3 Instalar en cada servidor a monitorear

En el servidor Windows, en una PowerShell como Administrador, con el instalador (install-agent.ps1) copiado a la máquina:

```
.\install-agent.ps1 -BackendUrl "https://noc.tudominio.com" `
-EnrollmentSecret "<el AGENT_ENROLLMENT_SECRET del .env central>"
```

Con ese único comando, el instalador:

- Descarga el .exe del agente desde el backend (si no hay uno local ya compilado al lado del script)
- Se auto-registra en el NOC llamando a POST /api/servers/enroll con el secreto compartido — el servidor aparece solo en el dashboard, sin tocar la base de datos ni el panel a mano
- Registra una Tarea Programada de Windows que corre como SYSTEM, con reinicio automático si el proceso se cae, y la arranca de inmediato

Para desinstalar: .\uninstall-agent.ps1 (opcionalmente con -RemoveFiles para borrar también los archivos instalados).

5. Seguridad: qué cubre y qué no

Ningún sistema es 100% seguro. Esta sección es una evaluación honesta de la postura de seguridad actual, para decidir con criterio si hace falta reforzar algo antes de usarlo con datos reales de un cliente.

Cubierto

Área	Protección implementada
Contraseñas y API keys	Hasheadas con bcrypt (costo 12), nunca en texto plano en la base
Autenticación	Login con email+contraseña; sesión vía JWT firmado guardado en una cookie httpOnly
Segundo factor (2FA)	TOTP (Google Authenticator, Authy, etc.) opcional por usuario, con códigos de respaldo de un solo uso
Almacenamiento de sesión	Cookie httpOnly + Secure (en producción) + SameSite=Strict — nunca accesible desde JavaScript
Revocación de sesión	Cada sesión queda registrada en la base: se puede cerrar individualmente (logout), en todos los dispositivos
CSRF	SameSite=Strict como primera barrera, más un chequeo de origen (Origin) en toda escritura como POST
Autorización (RBAC)	Roles ADMIN/ANALYST/VIEWER aplicados con middleware en cada endpoint sensible
Transporte	TLS vía Nginx + Let's Encrypt (una vez desplegado según la sección 3.6)
Rate limiting	Por IP en login, 2FA, telemetría, backups, endpoints admin, y un límite general sobre toda la API
Validación de entrada	Zod con modo estricto: rechaza campos no declarados en cada payload
Red / firewall	Postgres y los puertos internos de la app nunca expuestos a internet (ufw + binding a 127.0.0.1)
Auditoría	Registro de logins, 2FA, altas/bajas de usuarios y servidores, cambios de umbrales, ack/resolve de tickets
Credenciales por servidor	Cada agente tiene su propia API key (no un secreto compartido) — una fuga solo compromete un agente
Dependencias	0 vulnerabilidades reportadas por npm audit en backend y frontend (Next.js actualizado a la rama 12.0.1)

NO cubierto (limitaciones conocidas)

Área	Limitación
Auditoría externa	El código no fue revisado por nadie ajeno a este desarrollo, ni sometido a un pentest formal — ningún auditor externo
Multi-tenancy	Un backend = una organización, por diseño. Para Grupo Bistro como cliente único de este backend
Prueba de backup en Windows Server	Se creó un WMI (MSFT_WBSummary, Windows Server Backup) se revisó y se reforzaron sus logs
Recuperación de contraseña	No hay flujo de "olvidé mi contraseña" por email; solo un ADMIN puede recrear la cuenta

6. Operación diaria

6.1 Usuarios y servidores (pestaña Admin, solo rol ADMIN)

- **Nuevo usuario:** pestaña Admin → formulario "+ Nuevo usuario" (nombre, email, contraseña, rol).
- **Eliminar usuario:** botón Eliminar en su fila. No se puede eliminar la propia cuenta ni al último administrador restante.
- **Nuevo servidor (manual):** formulario "+ Nuevo servidor" — muestra el SERVER_ID y API_KEY una única vez, copiarlos al .env del agente.
- **Nuevo servidor (automático):** usar el instalador del agente con AGENT_ENROLLMENT_SECRET (sección 4.3) — no requiere pasar por el dashboard.
- **Rotar API key:** invalida la clave actual de inmediato y genera una nueva (útil si se sospecha que se filtró).

6.2 Umbrales por servidor

En Admin → Servidores → "Configurar": permite fijar umbrales de advertencia/crítico de CPU, RAM y disco distintos para cada servidor. Vacío = usar el default global (CPU 75/90%, RAM 80/90%, Disco 85/95%).

6.3 Ventanas de mantenimiento

Desde el mismo panel, botones de 1h/4h/24h (o "Terminar ahora"). Mientras está activa, la telemetría y el estado de backup se siguen guardando normalmente, pero no generan alertas — útil para actualizaciones planeadas sin ruido de falsas alarmas.

6.4 Notificaciones externas

En el .env central (sección 3.3), completar cualquiera de estos antes de levantar el stack:

- **Email:** SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_FROM, ALERT_EMAIL_TO
- **Slack:** SLACK_WEBHOOK_URL (Incoming Webhook)
- **Webhook genérico:** WEBHOOK_URL (Teams, Discord, PagerDuty, sistema propio)

Solo se notifican alertas HIGH o CRITICAL (ajustable con NOTIFY_MIN_SEVERITY).

6.5 Reconocer y resolver alertas

Pestaña Logs Regex: cada alerta abierta tiene botones "Reconocer" y "Resolver". Queda registrado quién y cuándo en la auditoría (Admin → Auditoría).

6.6 Segundo factor (2FA) y sesiones

Botón "Mi cuenta" (arriba a la derecha, en cualquier pantalla): cada usuario activa su propio 2FA escaneando un código QR con Google Authenticator, Authy o similar. Al confirmarlo se muestran 8 códigos de respaldo de un solo uso — guardarlos ahí mismo, no se vuelven a mostrar. A partir de ese momento, cada login pide el código además de la contraseña.

- **Perdiste el dispositivo del 2FA:** un ADMIN lo restablece desde Admin → Usuarios → "Restablecer 2FA" (lo desactiva y cierra las sesiones activas de esa cuenta).
- **Cerrar tu propia sesión en todos lados:** Mi cuenta → "Cerrar sesión en todos los dispositivos".
- **Forzar el cierre de sesión de otro usuario:** Admin → Usuarios → "Cerrar sesiones" (cuenta comprometida, baja de un empleado, etc.). Tiene efecto inmediato: a diferencia de un JWT clásico, la sesión queda invalidada en el momento, no cuando expira.

6.7 Deploys y rollback

```
./deploy/deploy.sh # actualiza a lo ultimo de la rama actual
./deploy/deploy.sh v1.2.0 # o a un commit/tag especifico
./deploy/rollback.sh <commit-anterior> # revierte el codigo (no las migraciones de BD)
```

7. Problemas conocidos y cómo se resolvieron

Bugs reales encontrados durante el desarrollo y las pruebas — documentados por si vuelven a aparecer.

Síntoma	Causa y solución
Acentos corruptos en el detalle de Hackup (vincleah)	El tipo de la consola de Windows no era fijo (mbcs ni el codepage OEM). Se resolvió cambiando a UTF-8.
"Could not parse schema engine response" en el Prisma de Stack Overflow	Los setters de Prisma no soportan SSL; la imagen node:20-alpine no lo trae por defecto. Se resolvió cambiando a node:20-alpine:ssl.
Scripts .sh fallan en el VPS Linux cuando se guardan	Se guardaban con saltos de línea CRLF (de Windows) en vez de LF. Se agregó .gitattributes para forzar LF.
Dashboard vacío al abrir una pestaña de WebSockets	El WebSocket no genera eventos nuevos desde que se conecta; no había carga del estado ya que se cargaba al conectar.
Build de Next.js falla con "specified property of object with values por defecto antes de un spread del mismo tipo — next dev no funciona"	El objeto con valores por defecto antes de un spread del mismo tipo — next dev no funciona.
WebSocket se cae al leer la cookie de sesión (cookie.parse versión 2 no pasó)	La cookie (cookie.parse versión 2 no pasó) a ser un paquete solo-ESM; al importarla desde CommonJS se cae.
Falla la generación de códigos 2FA (Dinamo reescribió el código) (ya no expone 'authenticator' como en versiones anteriores)	El código (Dinamo reescribió el código) (ya no expone 'authenticator' como en versiones anteriores).

8. Checklist final antes de anunciar la URL

- [] CORS_ORIGIN, NEXT_PUBLIC_API_URL y NEXT_PUBLIC_WS_URL apuntan al dominio real del cliente
- [] POSTGRES_PASSWORD, JWT_SECRET y AGENT_ENROLLMENT_SECRET son nuevos, generados para esta instalación (no reutilizados)
- [] NODE_ENV=production en el backend (ya viene fijo en docker-compose.yml) — de esto depende que la cookie de sesión salga marcada Secure
- [] Postgres y los puertos 3000/3001 no responden desde fuera del VPS (ufw status, y los puertos ligados a 127.0.0.1 en docker compose ps)
- [] TLS activo: candado verde en el navegador, wss:// (no ws://) en la consola
- [] Contraseña del usuario ADMIN inicial guardada en un lugar seguro (gestor de contraseñas, no en texto plano)
- [] 2FA activado en la cuenta ADMIN inicial, con los códigos de respaldo guardados en un lugar seguro
- [] Al menos un canal de notificación externa configurado (email, Slack o webhook) para alertas CRITICAL
- [] Estado de backup verificado con python agent.py --debug --once en al menos un servidor Windows real (sección 5, limitación conocida)
- [] Cliente/organización informado de las limitaciones de seguridad de la sección 5 (sin pentest externo, sin recuperación de contraseña por email)

9. Referencia rápida de comandos

Acción	Comando
Ver estado de los contenedores	<code>docker compose ps</code>
Ver logs del backend	<code>docker logs enterprise-soc-backend -f</code>
Ver logs del frontend	<code>docker logs enterprise-soc-frontend -f</code>
Reiniciar todo	<code>docker compose restart</code>
Crear un usuario/servidor de prueba	<code>docker exec enterprise-soc-backend node prisma/seed.js</code>
Backup manual de la base	<code>docker exec enterprise-soc-postgres pg_dump -U <user> nocsoc > backup.sql</code>
Deploy de una versión nueva	<code>./deploy/deploy.sh</code>
Rollback	<code>./deploy/rollback.sh <commit></code>
Instalar el agente en un servidor	<code>.\install-agent.ps1 -BackendUrl ... -EnrollmentSecret ...</code>
Desinstalar el agente	<code>.\uninstall-agent.ps1</code>
Probar el agente en modo debug (sin instalarlo)	<code>python agent.py --debug --once</code>

Documento generado como parte del desarrollo de Enterprise SOC. Mantenerlo actualizado a medida que el sistema evolucione.